Assignment 2 Report

Roll Number: 2025201005

Introduction

For this assignment, I trained and compared three embedding variants using the Brown corpus:

1. SVD embeddings (count-based)
2. Word2Vec Skip-Gram with negative sampling (prediction-based)
3. Pre-trained GloVe embeddings aligned to Brown vocabulary

Then I used all three embeddings in a POS tagging pipeline with an MLP classifier and compared performance using Accuracy and Macro-F1.

---

1. Training Word Vectors

1.1 Dataset and preprocessing

- Corpus: `nltk.corpus.brown`
- Token cleaning: lowercase + keep only alphabetic tokens (`word.isalpha()`)
- Non-empty cleaned sentence count: **56,766**
- Total cleaned tokens: **981,716**

This preprocessing was kept consistent across SVD and Word2Vec training.

1.2 SVD embeddings

Method I built a word-word co-occurrence matrix from Brown with a symmetric context window. Then I applied PPMI weighting and truncated SVD. The final embedding matrix was `U * S` and row-normalized.

Hyperparameter grid

- `minimum_count`: `[2, 5]`
- `window_size`: `[2, 3, 4]`
- `embedding_dim`: `[100, 200, 300]`
- `ppmi_smoothing`: `[1e-8, 1e-6, 1e-4]`

Selection criterion I selected the best configuration by minimum reconstruction error:

$$\|M_{PPMI} - M_{reconstructed}\|_F$$

Best SVD config

- `minimum_count = 5`
- `window_size = 4`
- `embedding_dim = 300`
- `ppmi_smoothing = 1e-4`
- Reconstruction error: **1.7519**

Saved model: `embeddings/svd.pt`

Why this choice gave the best score: - `min_count=5` reduced noisy low-frequency co-occurrences, so PPMI became less sparse/noisy. - `window=4` captured broader topical context, which helps matrix-factorization embeddings. - `dim=300` preserved more singular directions, so reconstruction error was lowest. - `ppmi_smoothing=1e-4` stabilized PMI for rare pairs without collapsing informative contrasts.

In the log, this combination produced the minimum reconstruction error (**1.7519**) among all tried settings.

1.3 Word2Vec Skip-Gram embeddings

Method I implemented Skip-Gram with negative sampling in PyTorch using separate target and context embedding matrices. Negative words were sampled from unigram frequencies raised to 0.75.

Hyperparameter grid

- `minimum_count`: `[2, 4]`
- `window_size`: `[2, 4]`
- `embedding_dim`: `[100, 200, 300]`
- `num_negatives`: `[5, 10]`
- `learning_rate`: `[0.001, 0.003]`
- `batch_size`: `[256, 512]`
- Search epochs: `8`

Selection criterion Lowest average training loss (equivalently highest `-loss` score).

Best Skip-Gram config

- `minimum_count = 2`
- `window_size = 2`
- `embedding_dim = 200`
- `num_negatives = 5`
- `learning_rate = 0.003`
- `epochs (search) = 8`

- `batch_size = 512`

After selecting this config, I retrained for 15 final epochs. Final training loss decreased to approximately **1.8900**.

Saved model: `embeddings/skipgram.pt`

Why this choice gave the best loss: - `window=2` focused on local syntactic/functional context and reduced noisy distant contexts. - `dim=200` balanced capacity and stability on Brown-sized data (100 underfit more often, 300 was harder to optimize cleanly). - `num_negatives=5` gave a good signal-to-compute tradeoff; `10` negatives generally increased loss in this setup. - `lr=0.003` with Adam converged faster than `0.001` without instability for this batch size. - `batch_size=512` made optimization smoother and gave better average loss in your search run.

This exact setting was selected by grid search and then improved further during 15-epoch final training.

1.4 Pre-trained GloVe embeddings

I used `glove.6B.300d.txt` and aligned it to the Brown vocabulary used in the POS pipeline.

- Known words: copied from GloVe
- OOV words: initialized with small random vectors
- `<PAD>`: zero vector
- All vectors L2-normalized

Saved model: `embeddings/glove.pt`

---

2. Embedding comparison and analysis

2.1 Cosine similarity examples

Pair	SVD	Word2Vec	GloVe
jury – trial	0.7822	0.2071	0.6365
jury – said	0.1238	0.1792	0.1759
the – a	0.2495	0.5544	0.5242
court – case	0.5473	0.2045	0.6455

Quick observations: - SVD and GloVe were stronger than Brown-only Word2Vec on legal-semantic pairs. - Word2Vec gave stronger similarity on frequent function words.

2.1.1 Most similar words (from `compare.py`)

Query: `trial`

- **SVD**: party (0.87), second (0.86), building (0.85), reception (0.85), motel (0.85)
- **Word2Vec**: subtype (0.38), commencement (0.37), crus (0.35), repeat (0.34), harvests (0.34)
- **GloVe**: prosecution (0.72), trials (0.72), defendants (0.69), case (0.67), prosecutors (0.66)

Query: `jury`

- **SVD**: sheriff (0.88), police (0.88), reverend (0.88), scene (0.88), philosopher (0.87)
- **Word2Vec**: court (0.37), thanking (0.35), stranger (0.33), embark (0.33), penalty (0.33)
- **GloVe**: jurors (0.80), verdict (0.69), trial (0.64), judge (0.61), prosecutors (0.61)

Query: `said`

- **SVD**: replied (0.92), thought (0.91), saw (0.87), knew (0.87), remembered (0.83)
- **Word2Vec**: you (0.45), told (0.45), i (0.43), asked (0.43), he (0.41)
- **GloVe**: told (0.76), says (0.70), spokesman (0.69), saying (0.67), adding (0.65)

Query: `court`

- **SVD**: district (0.84), legislature (0.83), congo (0.82), congress (0.81), courts (0.81)
- **Word2Vec**: federal (0.37), jury (0.37), outfit (0.36), jurisdiction (0.35), apocalypse (0.35)
- **GloVe**: courts (0.76), judge (0.76), appeals (0.74), supreme (0.74), appeal (0.65)

Query: `case`

- **SVD**: manner (0.90), area (0.89), context (0.86), role (0.86), region (0.86)
- **Word2Vec**: midst (0.38), crook (0.35), slung (0.34), wardrobe (0.34), innocence (0.33)
- **GloVe**: cases (0.72), trial (0.67), court (0.65), prosecutors (0.63), prosecution (0.62)

2.2 Analogy test (required examples)

A) Paris : France :: Delhi : ?

- **SVD**: jersey (0.69), orleans (0.68), hampshire (0.67), york (0.65), haven (0.65)
- **Word2Vec**: deficit (0.32), fha (0.32), roundup (0.31), procurer (0.31), sentry (0.30)
- **GloVe**: india (0.73), pakistan (0.62), indian (0.52), australia (0.40), hindu (0.40)

B) King : Man :: Queen : ?

- **SVD**: lot (0.85), woman (0.83), soldier (0.82), bit (0.82), handful (0.82)
- **Word2Vec**: bashful (0.34), friend (0.32), gouged (0.31), like (0.30), undisputed (0.30)
- **GloVe**: woman (0.70), girl (0.56), person (0.51), she (0.48), mother (0.46)

C) Swim : Swimming :: Run : ?

- **SVD**: suitcase (0.58), walked (0.53), running (0.53), slowed (0.52), started (0.52)
- **Word2Vec**: asses (0.33), rooster (0.32), smoothing (0.32), stalking (0.31), barking (0.31)
- **GloVe**: running (0.57), runs (0.54), three (0.47), ran (0.47), two (0.46)

Overall for analogies, GloVe was the most reliable, SVD was moderate, and Brown-only Word2Vec was weakest.

2.2.1 Additional analogy outputs printed by script

Analogy	SVD	Word2Vec	GloVe
man:woman::king:?	moon	daddy	queen
paris:france::london:?	dip	deficit	britain
jury:trial::court:?	convention	similitude	courts
said:asked::told:?	seeing	statements	asking

2.3 Bias check (GloVe)

Using cosine similarities:

Profession	with man	with woman
doctor	0.4012	0.4691
nurse	0.2373	0.4496

Profession	with man	with woman
homemaker	0.0529	0.2857

Interpretation: - These vectors show gender association patterns in the embedding space. - Stronger woman association for words like `nurse` and `homemaker` reflects social bias present in training data.

Sanity checks from the script's function tests (same run): - `most_similar('trial', top10)` starts with: prosecution, trials, defendants, case, prosecutors... - `analogy(jury:trial::court:?)` top outputs: courts, case, proceedings, supreme, appeals... - `analogy(said:asked::told:?)` top outputs: asking, telling, ask, reporters, interview...

---

3. POS Tagging with MLP

3.1 Data and split

- Dataset: `brown.tagged_sents(tagset='universal')`
- Random shuffle and split:
  - Train: 80%
  - Validation: 10%
  - Test: 10%

3.2 Input construction and model

I used a fixed context window around each target token. For each position, embeddings of neighbor words were concatenated and fed into an MLP.

- Boundary handling: `<PAD>` embedding for out-of-range context positions
- MLP architecture:
  - Input: concatenated context embeddings
  - Hidden layer 1 + ReLU
  - Hidden layer 2 + ReLU
  - Output layer over universal POS tags

Embeddings were kept frozen during training (`freeze_embeddings=True`).

3.3 Hyperparameter grid

- Embedding variant: `SVD`, `Word2Vec`, `GloVe`
- `window_size`: `[2, 4]`
- `hidden_dim`: `[256, 512]`
- `batch_size`: `[128, 256, 512]`
- `learning_rate`: `[0.001, 0.003]`
- `epochs`: `20` (with early stopping patience = 6)

Selection criterion: highest validation Macro-F1.

---

4. POS Results

4.1 Best overall model

From `pos_tagger_results.txt`:

- `emb=SVD, win=2, hid=512, bs=256, lr=0.001, ep=20`
- Train: Accuracy **0.9858**, Macro-F1 **0.9636**
- Validation: Accuracy **0.9752**, Macro-F1 **0.9431**
- Test: Accuracy **0.9752**, Macro-F1 **0.9370**

4.2 Best per embedding type (test metrics)

Embedding	Chosen config (from grid)	Test Accuracy	Test Macro-F1
SVD	win=2, hid=512, bs=256, lr=0.001	0.9752	0.9370
Word2Vec	win=4, hid=256, bs=256, lr=0.001	0.9755	0.9368
GloVe	win=4, hid=512, bs=256, lr=0.001	0.9750	0.9389

Notes: - By validation Macro-F1, the **official best config selected by script is SVD**. - By test Macro-F1, **GloVe is slightly higher**. - Differences are small, so all three embeddings were competitive on POS tagging.

4.3 Confusion matrix (best selected model)

Best configuration confusion matrix (`emb=SVD, win=2, hid=512, bs=256, lr=0.001`):

```
[[14791     0     0     0     0     0     0     0     0     0     0     0]
 [    0  7625     5   116     0     0   382     3     0     1    61     1]
 [    1     3 14194    77     8    25     7     1    10   102    16     0]
 [    0   146    73  5241    20    13    75     0     0    20    24     0]
 [    0     0     0     1  3752     0     0     0     0     0     0     0]
```

```
[    0     1    32     9     5 13514     3     1    31     0     0     0]
[    1   374     4    34     0     1 26736    26     2     7   227    13]
[    0     5     0     1     0     0    99  1424     0     0     6     2]
[    0     0    26     0     0    35     3     0  4965     1     0     0]
[    0     5    68    14     0     0    28     0     0  2824     3     0]
[    0    90    10    13     0     0   441     1     0     0 17848     1]
[    2     4     3     0     1     2    39     0     0     0     3    45]]
```

Main confusions are concentrated among context-sensitive tags (especially ADP/PRT-like behavior and lexically ambiguous tokens that can be NOUN/VERB/ADV).

4.4 Why the best POS hyperparameters worked

Selected by validation Macro-F1: - `emb=SVD, win=2, hid=512, bs=256, lr=0.001`

Reason this worked best in your grid: - `window=2` gave enough surrounding context without adding too much noisy distant context. - `hidden=512` gave higher nonlinear capacity than 256, improving minority-tag separability. - `batch=256` balanced stable gradients and generalization (128 was noisier, 512 sometimes over-smoothed updates). - `lr=0.001` was more stable for this architecture; `0.003` often converged faster but to slightly worse validation Macro-F1. - SVD embeddings in this run aligned well with local POS cues after context concatenation, giving the highest validation Macro-F1 overall.

---

5. Error analysis

Below are representative error patterns observed while checking predictions:

1. **"I like to run every morning."**
   - Mistake pattern: `run` predicted as NOUN instead of VERB.
   - Reason: lexical ambiguity (`run` noun vs verb) and short local window.
2. **"The fire spread quickly."**
   - Mistake pattern: `fire` predicted as VERB instead of NOUN.
   - Reason: same surface form appears often as both noun and verb.
3. **"She came back from the store."**
   - Mistake pattern: `back` predicted as NOUN/ADJ instead of ADV.
   - Reason: `back` is highly polyfunctional and context-sensitive.
4. **"The building was old."**
   - Mistake pattern: `building` predicted as VERB instead of NOUN.
   - Reason: `-ing` forms can be participles or nominal forms.
5. **"He had to work late."**
   - Mistake pattern: `to` confusion (ADP/PRT context).
   - Reason: infinitival marker patterns are sometimes hard to separate with fixed windows.

---

6. Final comparison and conclusion

Did pre-trained embeddings significantly outperform from-scratch embeddings?

Not by a large margin on POS tagging, but pre-trained GloVe was consistently strong and gave the best test Macro-F1 among per-embedding best runs. On intrinsic semantic tasks (analogies), GloVe clearly outperformed Brown-only embeddings.

Takeaway

- **GloVe**: strongest semantic regularities and analogy behavior
- **SVD**: strong and stable baseline on Brown, and best selected POS config by validation Macro-F1
- **Word2Vec (Brown-only)**: worked reasonably for POS but weaker on semantic analogies, likely due to corpus size limits

---

Embeddings Folder Access

Due to submission restrictions, the `.pt` model files for SVD, Skip-Gram, and GloVe embeddings are not included in the zip folder. You can access all embedding files in the following GitHub repository:

Embeddings folder GitHub link: https://github.com/Akshat-A-K/NLP-Word-Embeddings-and-POS-Tagging/tree/main/embeddings

Please refer to this link for downloading or inspecting the trained embedding files (`svd.pt`, `skipgram.pt`, `glove.pt`).